Machine Learning Lab (CS314)

Lab Record

Submitted To



Indian Institute of Information Technology Bhagalpur

Submitted in Partial Fulfilment of the Requirements for Machine Learning Lab Exam

Bachelor of Technology (B.Tech)
in
Computer Science and Engineering
From

Utsav Kumar (240101114)

Under the guidance of

Dr. Thejaswini M

Assistant Professor

Department of Computer Science & Engineering

IIIT BHAGALPUR, BIHAR 813210, INDIA

Aug-Dec 2026

List of Programs

1. Assignment 1 ……page 1
2. Assignment 2 ……page 3
3. Assignment 3 ……page 7
4. Assignment 4 ……page 17
5. Assignment 5 ……page 28
6. Assignment 6 ……page 31
7. Assignment 7 ……page 37
8. Assignment 8 ……page 40
9. Assignment 9 ……page 43
10. Assignment 10 …… page 45
11. Assignment 11……page 47

In [1]:
```
'''
Lab Assignment 1: Write a python program to compute Mean, Median, Mode, Variance,
Standard
Deviation without using any library or inbuilt functions.

'''
```

Out[1]:
```
'\nLab Assignment 1: Write a python program to compute Mean, Median, Mode, Varianc
e, Standard\nDeviation without using any library or inbuilt functions.\n\n'
```

In [2]:
```
#mean
def mean(data):
    if not data:
        return 0
    ttl_sum = sum(data)
    ttl_count = len(data)

    return (ttl_sum / ttl_count)
data = [0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9]
print("Mean :" , mean(data))
```

```
Mean : 4.5
```

In [3]:
```
#median
def median(data_raw):
    data = sorted(data_raw)
    n = len(data)
    if n % 2 == 0:
        mid1 = data[(n // 2) - 1]
        mid2 = data[n // 2]
        print("Even Size")
        return (mid1 + mid2) / 2
    print("Odd size")
    print(data)
    return data[n // 2]
data = [0 , 1 , 2 , 3 , 8 , 5 , 7 , 8 , 9]
print("Median :" , median(data))
data = [0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9]
print("Median :" , median(data))
data = [0 , 9 , 2 , 3 , 8 , 5 , 7 , 8 , 9]
print("Median :" , median(data))
```

```
Odd size
[0, 1, 2, 3, 5, 7, 8, 8, 9]
Median : 5
Even Size
Median : 4.5
Odd size
[0, 2, 3, 5, 7, 8, 8, 9, 9]
Median : 7
```

In [4]:
```
#varience
def varience(data):
    mean_val = mean(data)
    n = len(data)
```

```
    sums = 0
    for x in data:
        sums += (x - mean_val) * (x - mean_val) ;
    return (sums) / (n - 1)
data = [0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9]
print("Varience :" , varience(data))
```

```
Varience : 9.166666666666666
```

In [5]:
```
#standard deviation
def std_deviation(data):
    variance = varience(data)
    std_dev = variance ** 0.5
    return std_dev ;
data = [0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9]
print("Standard Deviation :" , std_deviation(data))
```

```
Standard Deviation : 3.0276503540974917
```

In [ ]:

In [ ]:

In [1]:
```
'''
Assignment 2: Study of Python Basic Libraries: Numpy

1. Create a 1D array from a Python list
2. Create a 2D array (matrix)
3. Perform element-wise operations (e.g., add 2 to every element)
4. Calculate the sum of all element
5. Create a 4x4 empty array and the shape is passed as a tuple (4, 4)
6. Create a 4x4 array filled with the value 5
7. Generate an array 10 random numbers sampled from standard normal distribution
8. Find dot product of two arrays (consider two 2x2 arrays)
9. Sort a NumPy array along rows, columns, as a whole
# by considering 3x3 array
10. Make a list of 3 numpy arrays and find the mean, median, variance and
standard deviation of all the arrays and output them as a list
11. Create a NumPy array of strings   (example: "[apple banana cherry mango
kiwi]") and split them (example out should be:

[list(['apple', 'banana', 'cherry', 'mango', 'kiwi'])]

'''
```

Out[1]:
```
'\nAssignment 2: Study of Python Basic Libraries: Numpy\n\n1. Create a 1D array fr
om a Python list \n2. Create a 2D array (matrix)\n3. Perform element-wise operatio
ns (e.g., add 2 to every element)  \n4. Calculate the sum of all element\n5. Creat
e a 4x4 empty array and the shape is passed as a tuple (4, 4)  \n6. Create a 4x4 a
rray filled with the value 5 \n7. Generate an array 10 random numbers sampled from
standard normal distribution  \n8. Find dot product of two arrays (consider two 2x
2 arrays)\n9. Sort a NumPy array along rows, columns, as a whole\n# by considering
3x3 array  \n10. Make a list of 3 numpy arrays and find the mean, median, variance
and standard deviation of all the arrays and output them as a list  \n11. Create a
NumPy array of strings   (example: "[apple banana cherry mango kiwi]") and split t
hem (example out should be: \n\n[list([\'apple\', \'banana\', \'cherry\', \'mango
\', \'kiwi\'])]\n\n'
```

In [2]:
```
import numpy as np
```

In [3]:
```
lst = [1 , 2 , 3 , 4 , 5]
arr = np.array(lst)
print(lst)
print(arr)
```

```
[1, 2, 3, 4, 5]
[1 2 3 4 5]
```

In [4]:
```
arr1 = np.array([[1 , 2 , 3] ,
                 [4 , 5 , 6] ,
                 [7 , 8 , 9]])
print(arr1)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

In [5]:
```
# adding 2 to arr
result = arr + 2
print("result on arr")
print(result)
#adding 2 to arr1
print("result on arr1 ")
result1 = arr1 + 2
print(result1)
```

```
result on arr
[3 4 5 6 7]
result on arr1
[[ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

In [6]:
```
# sum of all element on arr ;
print(f'Sum of arr : {np.sum(arr)}' )

# sum of all element on arr1 ;
print(f'Sum of arr1 : {np.sum(arr1)}' )
```

```
Sum of arr : 15
Sum of arr1 : 45
```

In [7]:
```
# 4x4 empty array
arr2 = np.empty((4 , 4))
print(arr2)
#since its empty it have garbage value
```

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

In [8]:
```
# 4x4 array with all val = 5
arr2 = np.full((4 , 4) , 5)
print(arr2)
```

```
[[5 5 5 5]
 [5 5 5 5]
 [5 5 5 5]
 [5 5 5 5]]
```

In [9]:
```
# random number array of size 10
arr3 = np.random.randn(10)
print(arr3)
```

```
[ 0.26810446 -0.20409707 -0.73470711  1.9120963  -0.08857837 -0.75066683
  2.50394889 -0.32644449 -1.56847285  0.00853156]
```

In [10]:
```
# dot product of of two 2 x 2 number array
arr4a = np.array([[1 , 2] ,
                  [3 , 4]])
arr4b = np.array([[5 , 6] ,
                  [7 , 8]])
```

```
result2 = np.dot(arr4a , arr4b)
print(f'dot product of two 2 x 2 array : \n{result2}')
```

```
dot product of two 2 x 2 array :
[[19 22]
 [43 50]]
```

In [11]:
```
# sorting an array along row , column and whole
# using arr1 as original array and making copy of it
arr5 = np.array([[1 , 12 , 3] ,
                 [14 , 15 , 6] ,
                 [7 , 18 , 9]])
print(f'Original array : \n{arr5} \n sorted one (whole) : \n{np.sort(arr5 , axis
= None)}')
print(f'Original array : \n{arr5} \n sorted one (row) : \n{np.sort(arr5 , axis =
0)}')
print(f'Original array : \n{arr5} \n sorted one (column) : \n{np.sort(arr5 , axis
= 1)}')
```

```
Original array :
[[ 1 12  3]
 [14 15  6]
 [ 7 18  9]]
 sorted one (whole) :
[ 1  3  6  7  9 12 14 15 18]
Original array :
[[ 1 12  3]
 [14 15  6]
 [ 7 18  9]]
 sorted one (row) :
[[ 1 12  3]
 [ 7 15  6]
 [14 18  9]]
Original array :
[[ 1 12  3]
 [14 15  6]
 [ 7 18  9]]
 sorted one (column) :
[[ 1  3 12]
 [ 6 14 15]
 [ 7  9 18]]
```

In [12]:
```
arr6 = [
    np.array([1 , 2 , 4]) ,
    np.array([3 , 9 , 5]) ,
    np.array([8 , 6 , 7])
]
mean_arr = [np.mean(a) for a in arr6]
median_arr = [np.median(a) for a in arr6]
varience_arr = [np.var(a) for a in arr6]
std_dev_arr = [np.std(a) for a in arr6]
print(f'mean for arr6 is : {mean_arr} \n median for arr6 is {median_arr} \n
varience for arr6 is {varience_arr} \n standard deviation for arr6 is
{std_dev_arr}')
```

```
mean for arr6 is : [2.3333333333333335, 5.666666666666667, 7.0]
 median for arr6 is [2.0, 5.0, 7.0]
 varience for arr6 is [1.5555555555555554, 6.222222222222222, 0.6666666666666666]
 standard deviation for arr6 is [1.247219128924647, 2.494438257849294, 0.81649658092
7726]
```

In [13]:
```
arr7 = np.array(["apple banana cherry mango kiwi"])
result3 = [arr7[0].split()]
print(result3)
```

```
[['apple', 'banana', 'cherry', 'mango', 'kiwi']]
```

In [14]:
```
# end of lab 2 assignment
```

In [1]:
```
'''
Assignment
3: Study of Python Basic Libraries: Pandas and Matplotlib
   !pip install pandas matplotlib
Pandas
1.  Write a code to load the dataset
# Make sure the 'iris_dataset.csv' file path is correct

2.  Display the first 5 rows of the DataFrame
3. Display descriptive statistics for the numerical columns
4.  Display a concise summary of the DataFrame (data types, non-null counts,
etc.)
5. Display the number of rows and columns(shape)
6 Display the count of each species type(class distribution)
7. Filter data by species for coloring

Matplotlib for Plots
-----------------------------------------------------------------------------
  #Single line plot
1. Plot 'SepalLengthCm' using a red dashed line ('r--')
-----------------------------------------------------------------------------
   #Multiline plot
   #Drop non-numeric columns like 'Id' and 'Species' for this type of plot
2. Plot all columns as separate lines
-----------------------------------------------------------------------------
  #Generates a scatter plot of Sepal Length vs. Petal Length, differentiating the
species by color.
3. Create the scatter plot
-----------------------------------------------------------------------------
   #Histogram of a Single Feature
4. plot Histogram of Sepal Width
-----------------------------------------------------------------------------
  #Box Plot of All Features
5. plot Box and whisker for each numerical attribute
'''
```

Out[1]:
```
"\nAssignment\n3: Study of Python Basic Libraries: Pandas and Matplotlib\n   !pip
install pandas matplotlib\nPandas\n1.  Write a code to load the dataset\n# Make su
re the 'iris_dataset.csv' file path is correct\n\n2.  Display the first 5 rows of
the DataFrame\n3. Display descriptive statistics for the numerical columns  \n4.
Display a concise summary of the DataFrame (data types, non-null counts, etc.)\n5.
Display the number of rows and columns(shape)\n6 Display the count of each species
type(class distribution)\n7. Filter data by species for coloring\n\nMatplotlib for
Plots\n_____
_____\n  #Single line plot\n1. Plot 'SepalLengthCm'
using a red dashed line ('r--')  \n_____
_____\n   #Multili
ne plot \n   #Drop non-numeric columns like 'Id' and 'Species' for this type of pl
ot  \n2. Plot all columns as separate lines\n_____
_____\n  #G
enerates a scatter plot of Sepal Length vs. Petal Length, differentiating the spec
ies by color. \n3. Create the scatter plot  \n_____
_____
\n   #Histogram of a Single Feature  \n4. plot Histogram of Sepal Width  \n_____
_____
_____\n  #Box Plot of All Features \n5. plot Box and whiske
r for each numerical attribute\n"
```

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

import warnings
```

In [3]:
```
# data import
df = pd.read_csv('Iris.csv')

df
```

Out[3]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

In [4]:
```
# 1st 5 row
df.head()
```

Out[4]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

In [5]:
```
# 3. Display descriptive statistics for the numerical columns
df.describe
```

Out[5]:
```
<bound method NDFrame.describe of       Id  SepalLengthCm  SepalWidthCm  PetalLeng
thCm  PetalWidthCm  \
0      1            5.1           3.5            1.4           0.2
1      2            4.9           3.0            1.4           0.2
2      3            4.7           3.2            1.3           0.2
3      4            4.6           3.1            1.5           0.2
4      5            5.0           3.6            1.4           0.2
..   ...            ...           ...            ...           ...
145  146            6.7           3.0            5.2           2.3
146  147            6.3           2.5            5.0           1.9
147  148            6.5           3.0            5.2           2.0
148  149            6.2           3.4            5.4           2.3
149  150            5.9           3.0            5.1           1.8

            Species
0       Iris-setosa
1       Iris-setosa
2       Iris-setosa
3       Iris-setosa
4       Iris-setosa
..              ...
145  Iris-virginica
146  Iris-virginica
147  Iris-virginica
148  Iris-virginica
149  Iris-virginica

[150 rows x 6 columns]>
```

In [6]:
```
# Display a concise summary of the DataFrame (data types, non-null counts, etc.)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

In [7]:
```
# Display the number of rows and columns(shape)
df.shape
```

Out[7]:
```
(150, 6)
```

In [8]:
```
# Display the count of each species type(class distribution)
print(df["Species"].value_counts)
print()
print(df["Species"].value_counts())
```

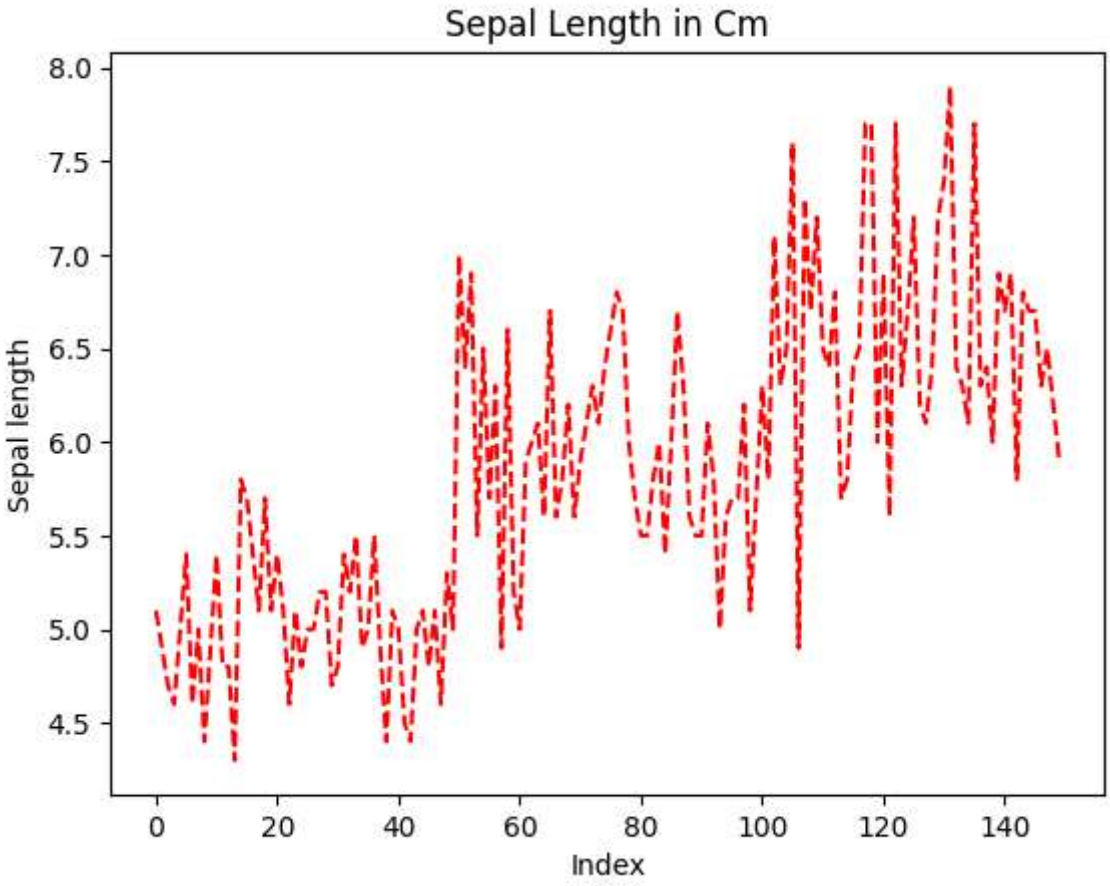
```
<bound method IndexOpsMixin.value_counts of 0         Iris-setosa
1         Iris-setosa
2         Iris-setosa
3         Iris-setosa
4         Iris-setosa
           ...
145    Iris-virginica
146    Iris-virginica
147    Iris-virginica
148    Iris-virginica
149    Iris-virginica
Name: Species, Length: 150, dtype: object>

Species
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

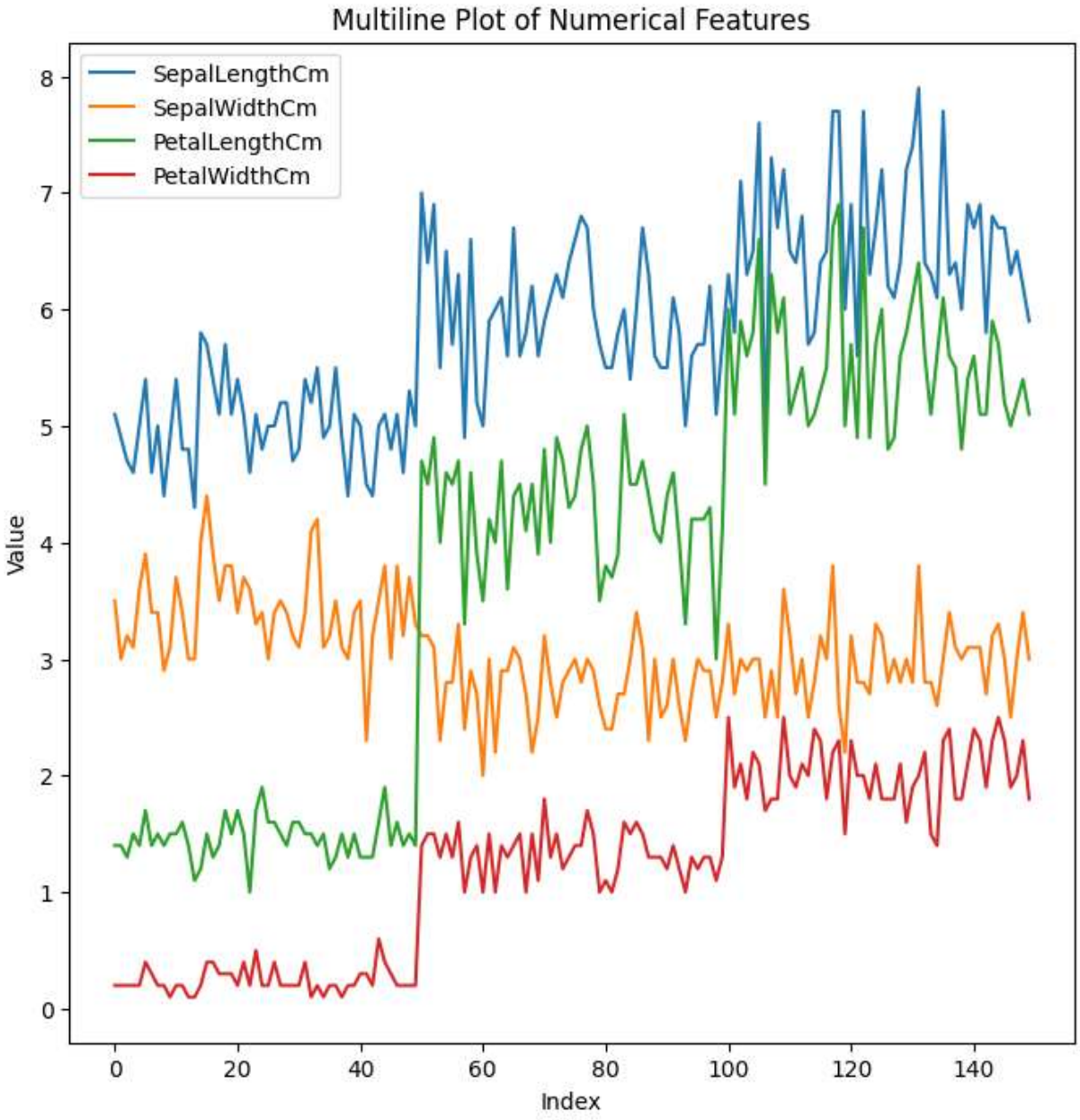
In [9]:
```
setosa = df[df["Species"] == "Iris-setosa"]
versicolor = df[df["Species"] == "Iris-versicolor"]
virginica = df[df["Species"] == "Iris-virginica"]
```

In [10]:
```
# Matplotlib for Plots
#
_____________________________________________________________________
#   #Single line plot
# 1. Plot 'SepalLengthCm' using a red dashed line ('r--')
#
_____________________________________________________________________
#    #Multiline plot
#    #Drop non-numeric columns like 'Id' and 'Species' for this type of plot
# 2. Plot all columns as separate lines
#
_____________________________________________________________________
#   #Generates a scatter plot of Sepal Length vs. Petal Length, differentiating
the species by color.
# 3. Create the scatter plot
#
_____________________________________________________________________
#    #Histogram of a Single Feature
# 4. plot Histogram of Sepal Width
#
_____________________________________________________________________
#   #Box Plot of All Features
# 5. plot Box and whisker for each numerical attribute
```

In [11]:
```
plt.plot(df['SepalLengthCm'] , 'r--')
plt.title("Sepal Length in Cm")
plt.xlabel("Index")
plt.ylabel("Sepal length")
plt.show()
```



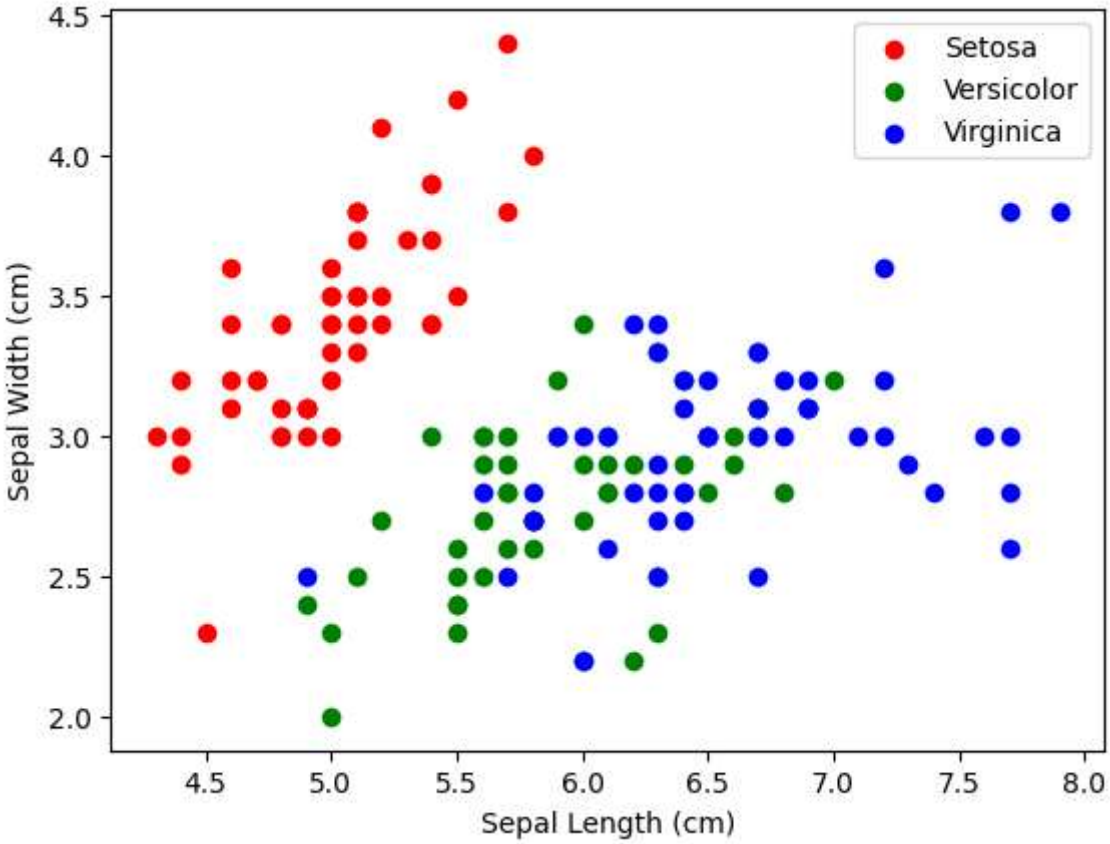
In [12]:
```
numeric_df = df.drop(columns=["Id" , "Species"])
numeric_df.plot(figsize=(8 , 8))
plt.title("Multiline Plot of Numerical Features")
plt.xlabel("Index")
plt.ylabel("Value")
plt.show()
```



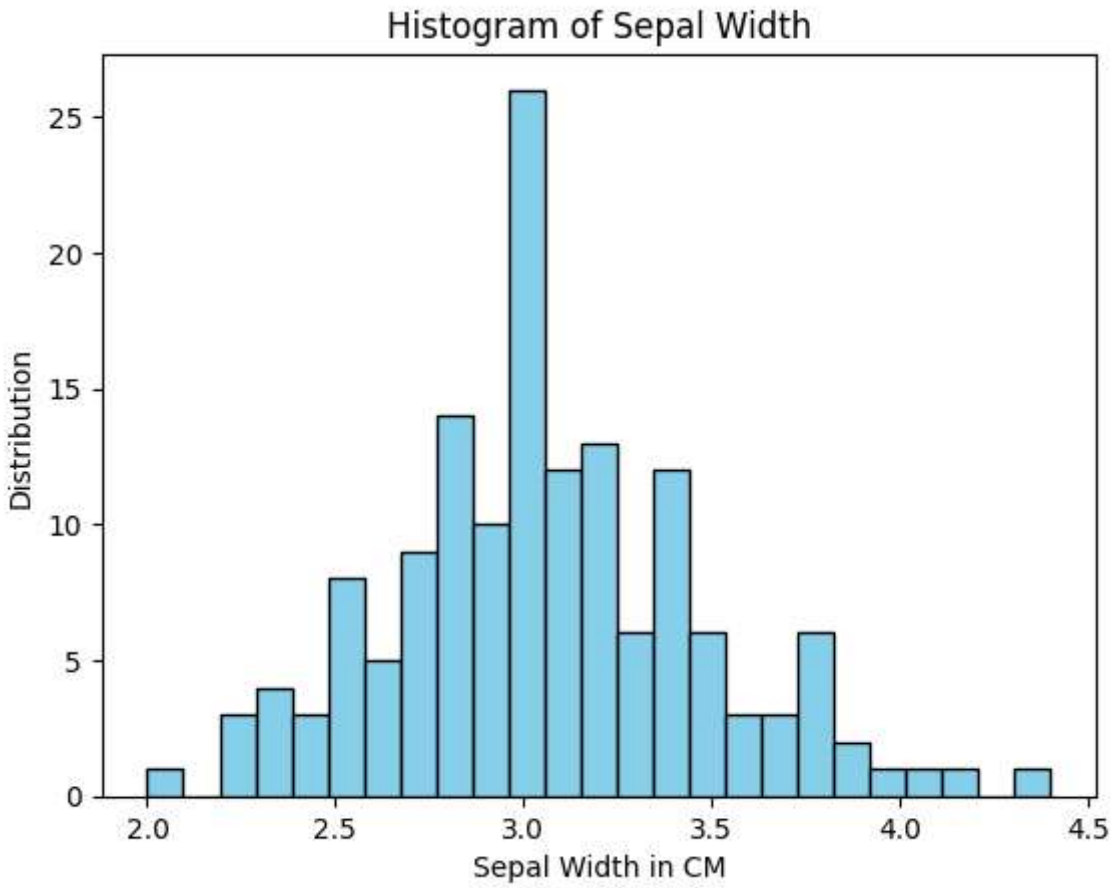
In [13]:
```
# setosa = df[df["Species"] == "Iris-setosa"]
# versicolor = df[df["Species"] == "Iris-versicolor"]
# virginica = df[df["Species"] == "Iris-virginica"]

#  Filter data by species for coloring
plt.scatter(setosa["SepalLengthCm"], setosa["SepalWidthCm"], color="red",
label="Setosa")
plt.scatter(versicolor["SepalLengthCm"], versicolor["SepalWidthCm"],
color="green", label="Versicolor")
plt.scatter(virginica["SepalLengthCm"], virginica["SepalWidthCm"], color="blue",
label="Virginica")

plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
plt.legend()
plt.show()
```



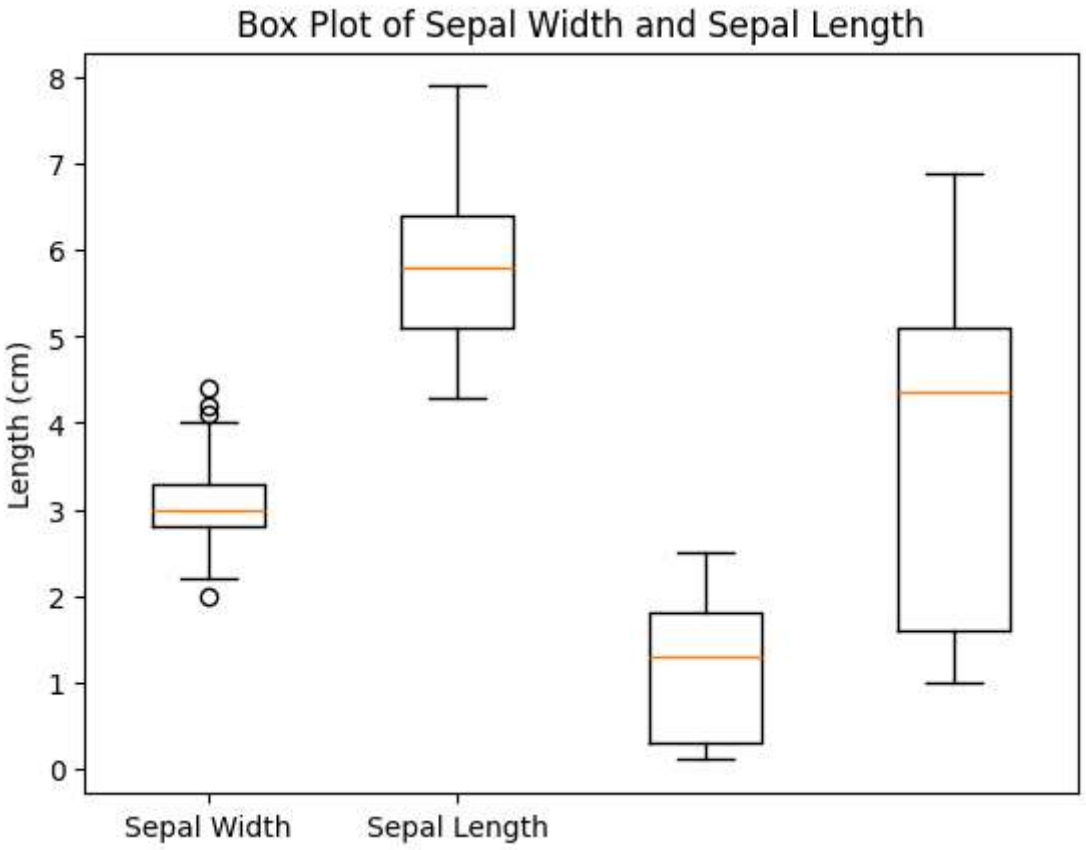
In [14]:
```
plt.hist(df["SepalWidthCm"] , bins=25, edgecolor="black" , color="skyblue")
plt.title("Histogram of Sepal Width")
plt.xlabel("Sepal Width in CM")
plt.ylabel("Distribution")
plt.show()
```



In [15]:
```
import matplotlib.pyplot as plt

plt.boxplot([
    df["SepalWidthCm"],
    df["SepalLengthCm"],
    df["PetalWidthCm"],
    df["PetalLengthCm"]
])

plt.title("Box Plot of Sepal Width and Sepal Length")
plt.xticks([1, 2], ["Sepal Width", "Sepal Length"])
plt.ylabel("Length (cm)")
plt.show()
```


Box Plot of Sepal Width and Sepal Length
Length (cm)
0
1
2
3
4
5
6
7
8
Sepal Width
Sepal Length

In [ ]:

In [1]:
```
'''
4. For the given datasets  (attached USA_Housing.csv file) apply linear
regression model and evaluate the same.
Note: Dataset is attached, no need to download from kaggle)
 '''
```

Out[1]:
```
'\n4. For the given datasets  (attached USA_Housing.csv file) apply linear regress
ion model and evaluate the same.\nNote: Dataset is attached, no need to download f
rom kaggle)\n '
```

In [2]:
```
# Data
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Split data
from sklearn.model_selection import train_test_split

#model train
from sklearn.linear_model import LinearRegression

#model Evaluation
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

#randon forest regressor

from sklearn.ensemble import RandomForestRegressor
```

In [3]:
```
data = pd.read_csv('USA_Housing - USA_Housing.csv')
data.head()
```

Out[3]:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	Ad
0	79545.45857	5.682861	7.009188	4.09	23086.80050	1.059034e+06	208 Michael Fer 674\nLaurabu
1	79248.64245	6.002900	6.730821	3.09	40173.07217	1.505891e+06	188 Johnson Suite 079 Kathlee
2	61287.06718	5.865890	8.512727	5.13	36882.15940	1.058988e+06	9127 Eli Stravenue\nDanie WI 0
3	63345.24005	7.188236	5.586729	3.26	34310.24283	1.260617e+06	USS Barnett\nF
4	59982.19723	5.040555	7.839388	4.23	26354.10947	6.309435e+05	USNS Raymond AE

In [4]:
```
# target value is Price

# variables are avg home age , avg no of room , avg no of bedsrooms , area population , address
```

In [5]:
```
data.columns
```

Out[5]:
```
Index(['Avg. Area Income', 'Avg. Area House Age', 'Avg. Area Number of Rooms',
       'Avg. Area Number of Bedrooms', 'Area Population', 'Price', 'Address'],
      dtype='object')
```

In [6]:
```
dependent_list = ["Price"]

independent_list = [
    "Avg. Area Income",
    "Avg. Area House Age",
    "Avg. Area Number of Rooms",
    "Avg. Area Number of Bedrooms",
    "Area Population"
]

for col in independent_list:
    plt.figure(figsize=(7, 4))

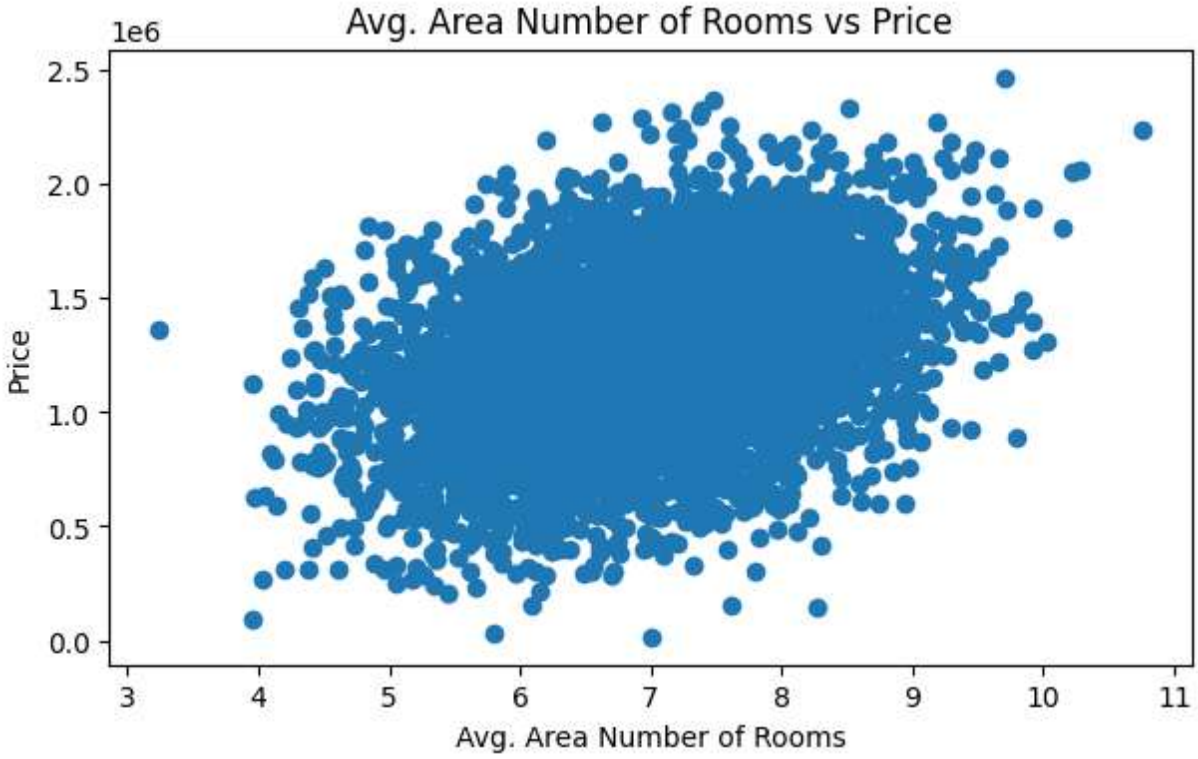
    plt.scatter(data[col], data["Price"])

    plt.xlabel(col)
    plt.ylabel("Price")
    plt.title(f"{col} vs Price")
```

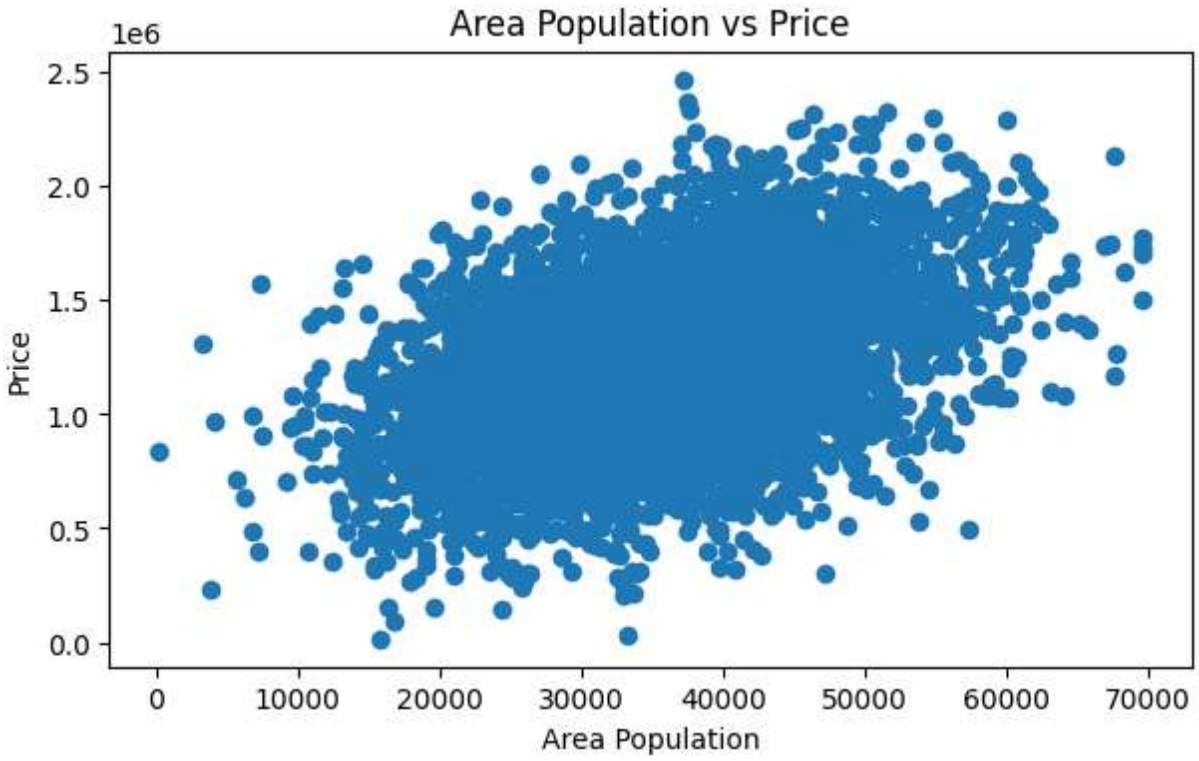
```
plt.show()
```


Avg. Area Income vs Price
1e6
2.5
2.0
1.5
1.0
0.5
0.0
Price
20000
40000
60000
80000
100000
Avg. Area Income


Avg. Area House Age vs Price
1e6
2.5
2.0
1.5
1.0
0.5
0.0
Price
3
4
5
6
7
8
9
Avg. Area House Age


Avg. Area Number of Rooms vs Price
1e6
2.5
2.0
1.5
1.0
0.5
0.0
Price
3
4
5
6
7
8
9
10
11
Avg. Area Number of Rooms

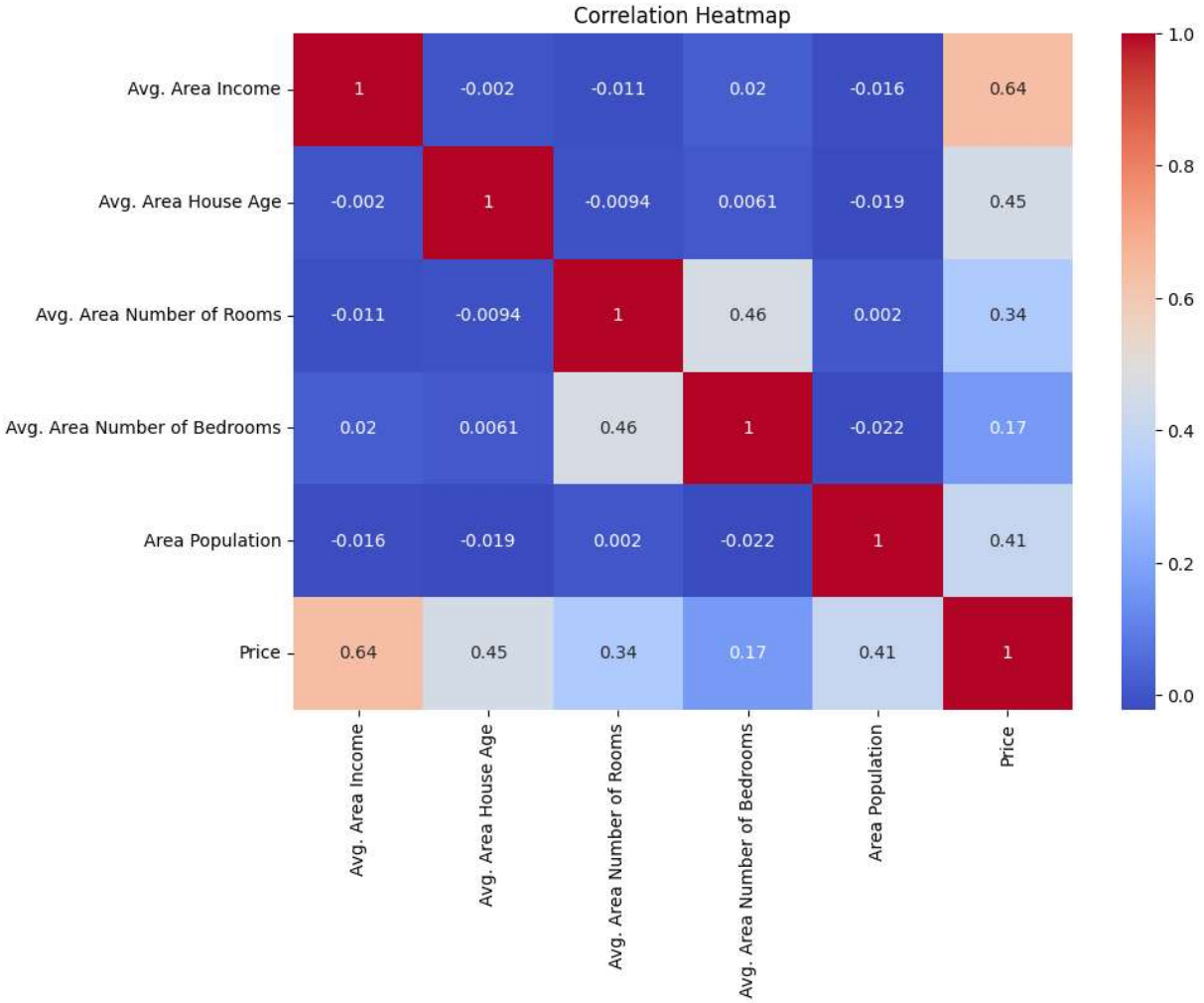

Avg. Area Number of Bedrooms vs Price
1e6
2.5
2.0
1.5
1.0
0.5
0.0
Price
2
3
4
5
6
Avg. Area Number of Bedrooms



In [7]:
```
# heat map of dependence
plt.figure(figsize=(10, 7))

sns.heatmap(
    data[independent_list + ["Price"]].corr(),
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")
plt.show()
```



In [8]:
```
# data correctness
data.isnull().sum()
```

Out[8]:
```
Avg. Area Income              0
Avg. Area House Age           0
Avg. Area Number of Rooms     0
Avg. Area Number of Bedrooms  0
Area Population               0
Price                         0
Address                       0
dtype: int64
```

In [9]:
```
data.duplicated().sum()
```

Out[9]:
```
0
```

In [10]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 7 columns):
 #   Column                        Non-Null Count  Dtype  
---  ------                        --------------  -----  
 0   Avg. Area Income              5000 non-null   float64
 1   Avg. Area House Age           5000 non-null   float64
 2   Avg. Area Number of Rooms     5000 non-null   float64
 3   Avg. Area Number of Bedrooms  5000 non-null   float64
 4   Area Population               5000 non-null   float64
 5   Price                         5000 non-null   float64
 6   Address                       5000 non-null   object 
dtypes: float64(6), object(1)
memory usage: 273.6+ KB
```

In [11]:
```
data[independent_list].describe()
```

Out[11]:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000
mean	68583.108984	5.977222	6.987792	3.981330	36163.516039
std	10657.991214	0.991456	1.005833	1.234137	9925.650114
min	17796.631190	2.644304	3.236194	2.000000	172.610686
25%	61480.562390	5.322283	6.299250	3.140000	29403.928700
50%	68804.286405	5.970429	7.002902	4.050000	36199.406690
75%	75783.338665	6.650808	7.665871	4.490000	42861.290770
max	107701.748400	9.519088	10.759588	6.500000	69621.713380

In [12]:
```
#data price describe
data["Price"].describe
```

Out[12]:
```
<bound method NDFrame.describe of 0       1.059034e+06
1       1.505891e+06
2       1.058988e+06
3       1.260617e+06
4       6.309435e+05
            ...
4995    1.060194e+06
4996    1.482618e+06
4997    1.030730e+06
4998    1.198657e+06
4999    1.298950e+06
Name: Price, Length: 5000, dtype: float64>
```

In [13]:
```
x = data[independent_list]

y = data["Price"]
```

```
# split data
```

In [14]:
```
# train test split
x_train , x_test , y_train , y_test = train_test_split(
    x , y ,
    test_size=0.2 , random_state=42
)
```

In [15]:
```
# using linear regration
model = LinearRegression()

model.fit(x_train , y_train)
```

Out[15]:
```
▾ LinearRegression
LinearRegression()
```

In [16]:
```
# test model on data which are not seen
y_pred = model.predict(x_test)
```

In [17]:
```
# for regression

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("R2 Score:", r2)
```

```
MAE: 80879.09722218332
MSE: 10089009299.49946
R2 Score: 0.9179971706985317
```

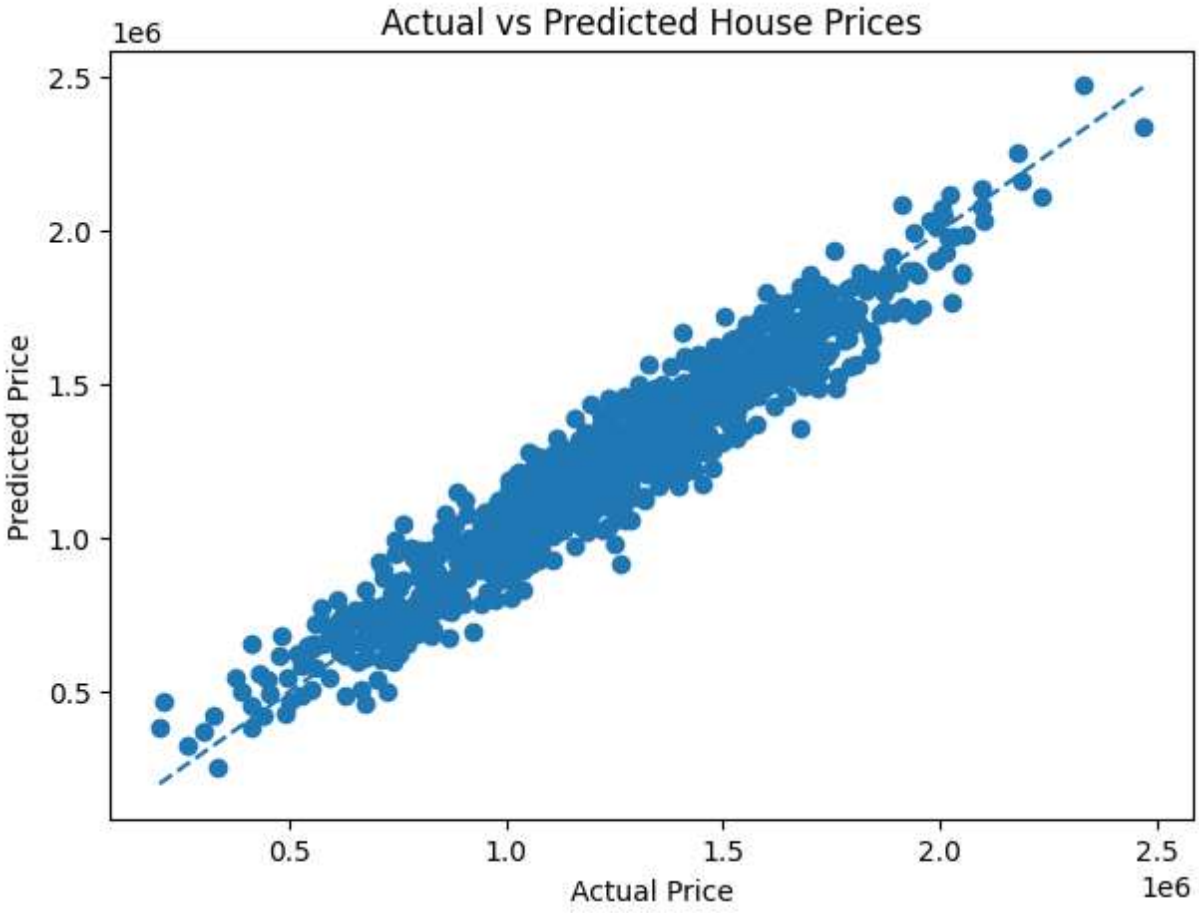
In [18]:
```
# visualize actual vss predicted
plt.figure(figsize=(7, 5))

plt.scatter(y_test, y_pred)

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle="--"
)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted House Prices")

plt.show()
```



In [19]:
```python
# create a model for random forest regressor

# basic configuration
# rf_model = RandomForestRegressor(
#     n_estimators=100,
#     random_state=42
# )

# stronger configuration
rf_model = RandomForestRegressor(
    n_estimators=30000,
    max_depth=None,
    min_samples_split=2,
    random_state=42,
    n_jobs=-1
)
```

In [20]:
```python
# train that model on our existing data set
rf_model.fit(x_train, y_train)
```

Out[20]:

RandomForestRegressor

```
RandomForestRegressor(n_estimators=30000, n_jobs=-1, random_state=42)
```

In [21]:
```
# predict values on our test data set
rf_pred = rf_model.predict(x_test)
```

In [22]:
```
# evaluate the random forest model
rf_mae = mean_absolute_error(y_test, rf_pred)
rf_mse = mean_squared_error(y_test, rf_pred)
rf_rmse = np.sqrt(rf_mse)
rf_r2 = r2_score(y_test, rf_pred)

print("MAE:", rf_mae)
print("MSE:", rf_mse)
print("RMSE:", rf_rmse)
print("R2 Score:", rf_r2)
```

```
MAE: 93883.13872981176
MSE: 14253590950.327282
RMSE: 119388.40375148368
R2 Score: 0.8841477145143845
```

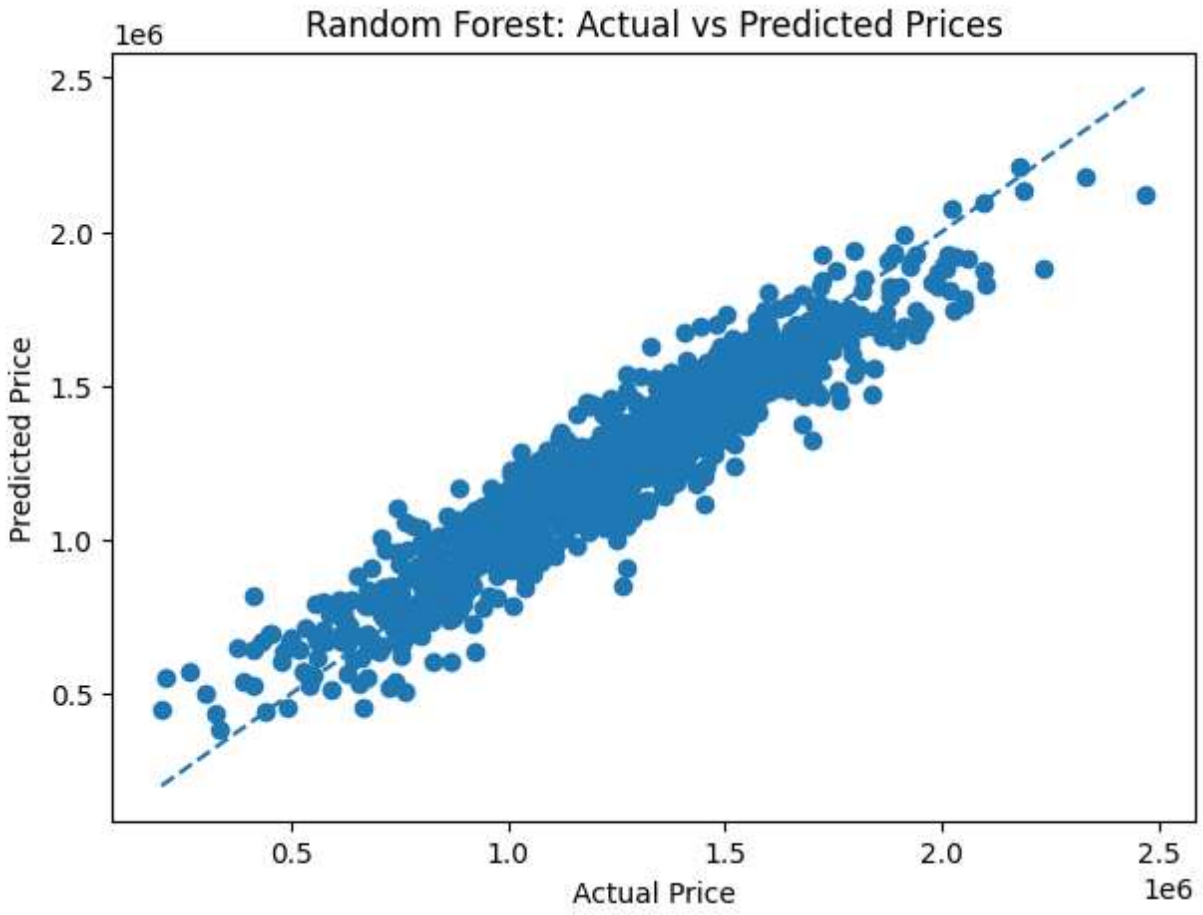
In [23]:
```
# actual vs predicted for rf
plt.figure(figsize=(7, 5))

plt.scatter(y_test, rf_pred)

# Perfect prediction line
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle="--"
)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Random Forest: Actual vs Predicted Prices")

plt.show()
```



In [24]:
```
# input form user

new_house = pd.DataFrame([{
    "Avg. Area Income": 70000,
    "Avg. Area House Age": 6.0,
    "Avg. Area Number of Rooms": 7.0,
    "Avg. Area Number of Bedrooms": 4.0,
    "Area Population": 35000
}])

prediction = model.predict(new_house)

print("Predicted House Price:", prediction[0])
```

```
Predicted House Price: 1250170.952972008
```

In [ ]:

In [1]:
```
'''
Write a ML code to classify (binary classification) iris flower into two classes
iris-setosa and iris-versicolor using logistic regression and evaluate the same
considering the following parameters:
Accuracy,
Error Rate
Kappa,
Sensitivity,
Precision
Recall
F fall
confusion matrix.
'''
```

Out[1]:
```
'\nWrite a ML code to classify (binary classification) iris flower into two classe
s iris-setosa and iris-versicolor using logistic regression and evaluate the same
considering the following parameters:\nAccuracy,\nError Rate\nKappa,\nSensitivity,
\nPrecision\nRecall\nF fall\nconfusion matrix.\n'
```

In [2]:
```
# importing important libraries

import pandas as pd
import numpy as np

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    cohen_kappa_score
)
```

In [3]:
```
iris = pd.read_csv('iris_data.csv')
```

In [4]:
```
iris.head()
iris.head()

print(iris['species'].value_counts())

# Features (all columns except species)
X = iris.drop("species", axis=1)

# Target column
y = iris["species"]
```

```
species
setosa        50
versicolor    50
Name: count, dtype: int64
```

In [5]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

# importing the model
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
```

Out[5]:
```
LogisticRegression
LogisticRegression(max_iter=200)
```

In [6]:
```
# predicting the target variable for test data
y_pred = model.predict(X_test)
```

In [7]:
```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)

# Error Rate
error_rate = 1 - accuracy

# Kappa Score
kappa = cohen_kappa_score(y_test, y_pred)

# Precision, Recall, Sensitivity, F1
precision = precision_score(y_test, y_pred, pos_label="versicolor")
recall = recall_score(y_test, y_pred, pos_label="versicolor")
sensitivity = recall          # Sensitivity = Recall
f1 = f1_score(y_test, y_pred, pos_label="versicolor")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred, labels=["setosa", "versicolor"])

# TP, TN, FP, FN
tn, fp, fn, tp = cm.ravel()

print("Accuracy      :", accuracy)
print("Error Rate    :", round(error_rate, 4))
print("Kappa Score   :", round(kappa, 4))
print("Sensitivity   :", round(sensitivity, 4))
print("Precision     :", round(precision, 4))
print("Recall        :", round(recall, 4))
print("F1 Score      :", round(f1, 4))

print("\nConfusion Matrix:")
print(cm)

print("\nTrue Negative (TN):", tn)
print("False Positive (FP):", fp)
print("False Negative (FN):", fn)
print("True Positive (TP):", tp)
```

```
Accuracy      : 1.0
Error Rate    : 0.0
Kappa Score   : 1.0
Sensitivity   : 1.0
Precision     : 1.0
Recall        : 1.0
F1 Score      : 1.0

Confusion Matrix:
[[10  0]
 [ 0 10]]

True Negative (TN): 10
False Positive (FP): 0
False Negative (FN): 0
True Positive (TP): 10
```

In [ ]:

In [1]:
```
'''
16. Write a ML code for iris datasets classifications using sklearn svm library
(3 separate programs)
Dataset attached consists of 2, features and 2 classes (target) names as class 0
and class 1.

a. Using SVM linear kernel, print accuracy and loss, and plot decision boundary
b. Using SVM poly kernel print accuracy and loss, and plot decision boundary
c. Using SVM Guassian RBF kernel print accuracy and loss, and plot decision
boundary

Task on Thursday: 27-08-2026


Tutorial: Below two programs are not to be included as Lab assignment and it is
just for enhancing practical knowledge
1. Use SVM for solving any regression problem
2. Caution: (This will be explained on Thursday evening theory class and not to
be considered  for lab assignment. Just for enhancing the knowledge of ML
practical).
Develop a Python program in Jupyter Notebook to classify
images into Cat and Dog classes using a Support Vector Machine
(SVM) classifier. The program should:
Load
cat and dog images from separate folders.
Resize
the images to a fixed size.
Extract
HOG (Histogram of Oriented Gradients) features from the images.
Divide
the dataset into training and testing sets.
Train
an SVM classifier using the training data.
Predict
the class of the test images.
Calculate
and display the classification accuracy and classification report.
Test
the trained model on a new image and display whether it is a Cat or Dog
'''
```

Out[1]:
```
'\n16. Write a ML code for iris datasets classifications using sklearn svm library
(3 separate programs)\nDataset attached consists of 2, features and 2 classes (tar
get) names as class 0 and class 1.\n\na. Using SVM linear kernel, print accuracy a
nd loss, and plot decision boundary\nb. Using SVM poly kernel print accuracy and l
oss, and plot decision boundary\nc. Using SVM Guassian RBF kernel print accuracy a
nd loss, and plot decision boundary\n\nTask on Thursday: 27-08-2026\n\n\nTutorial:
Below two programs are not to be included as Lab assignment and it is just for enh
ancing practical knowledge\n1. Use SVM for solving any regression problem \n2. Cau
tion: (This will be explained on Thursday evening theory class and not to be consi
dered  for lab assignment. Just for enhancing the knowledge of ML practical).\nDev
elop a Python program in Jupyter Notebook to classify\nimages into Cat and Dog cla
sses using a Support Vector Machine\n(SVM) classifier. The program should:\nLoad\n
cat and dog images from separate folders.\nResize\nthe images to a fixed size.\nEx
tract\nHOG (Histogram of Oriented Gradients) features from the images.\nDivide\nth
e dataset into training and testing sets.\nTrain\nan SVM classifier using the trai
ning data.\nPredict\nthe class of the test images.\nCalculate\nand display the cla
ssification accuracy and classification report.\nTest\nthe trained model on a new
image and display whether it is a Cat or Dog\n'
```

In [2]:
```python
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, hinge_loss

# Load dataset
data = pd.read_csv("iris_dataset_SVM - iris_dataset_SVM.csv")

# Features and Target
X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Support Vector Machine Linear Kernal

In [3]:
```python
# Linear Kernel SVM
model = SVC(kernel="linear")
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
loss = hinge_loss(y_test, model.decision_function(X_test))

print("Linear Kernel")
print("Accuracy:", accuracy)
print("Loss:", loss)
```

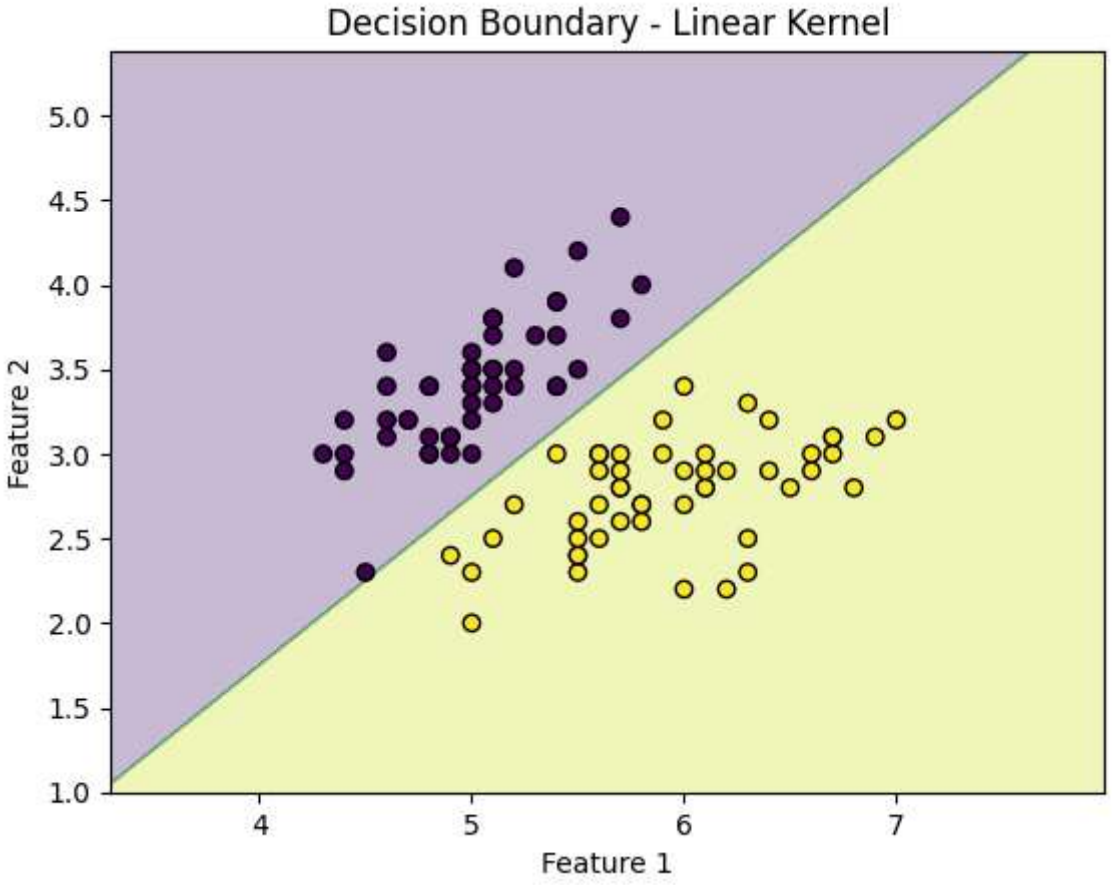
```
# Decision Boundary
x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
y_min, y_max = X[:,1].min()-1, X[:,1].max()+1

xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                     np.arange(y_min, y_max, 0.02))

Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(X[:,0], X[:,1], c=y, edgecolors="k")
plt.title("Decision Boundary - Linear Kernel")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

```
Linear Kernel
Accuracy: 1.0
Loss: 0.033334255562280465
```



Support Vector Machine Polynomial Kernal

In [4]:
```
# Polynomial Kernel SVM
model = SVC(kernel="poly", degree=3)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
```

```
loss = hinge_loss(y_test, model.decision_function(X_test))

print("Polynomial Kernel")
print("Accuracy:", accuracy)
print("Loss:", loss)

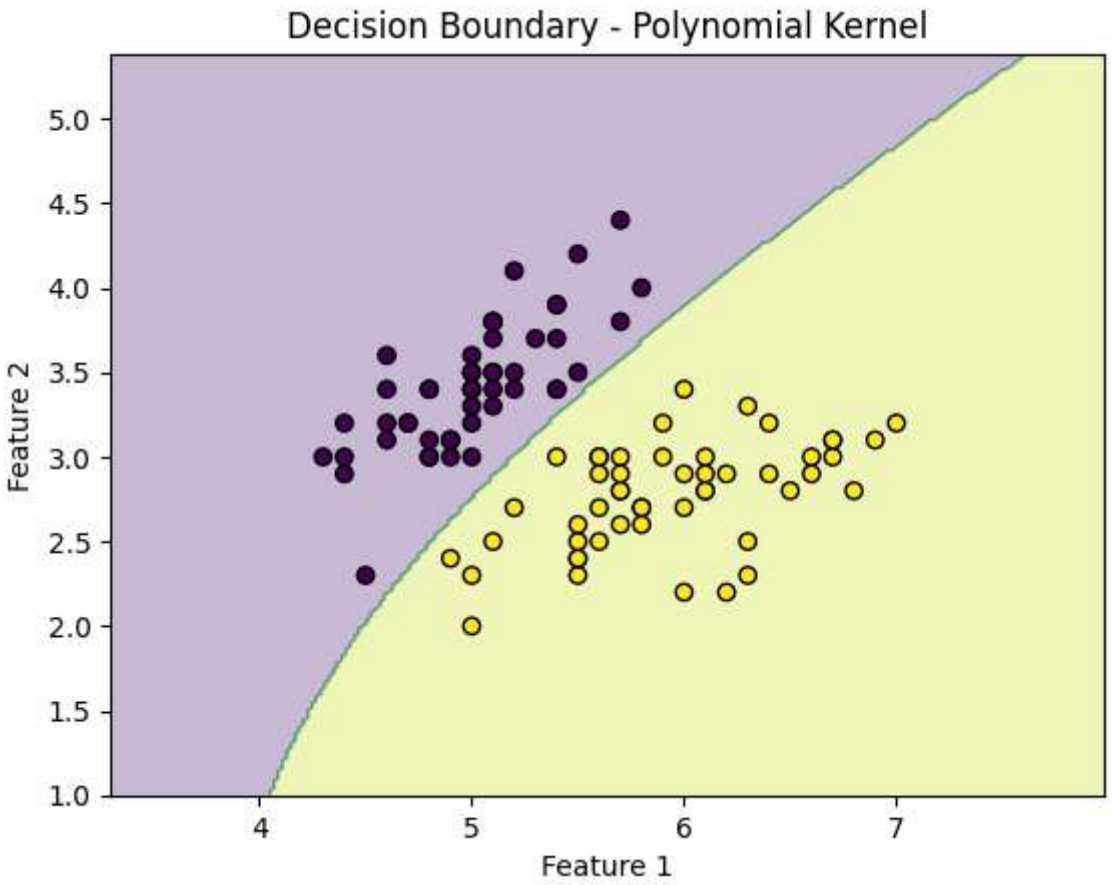
# Decision Boundary
x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
y_min, y_max = X[:,1].min()-1, X[:,1].max()+1

xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                     np.arange(y_min, y_max, 0.02))

Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(X[:,0], X[:,1], c=y, edgecolors="k")
plt.title("Decision Boundary - Polynomial Kernel")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

```
Polynomial Kernel
Accuracy: 1.0
Loss: 0.0
```



Support Vector Machine Gaussian RBF Kernal

In [5]:
```
# Gaussian RBF Kernel SVM
model = SVC(kernel="rbf")
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
loss = hinge_loss(y_test, model.decision_function(X_test))

print("Gaussian RBF Kernel")
print("Accuracy:", accuracy)
print("Loss:", loss)

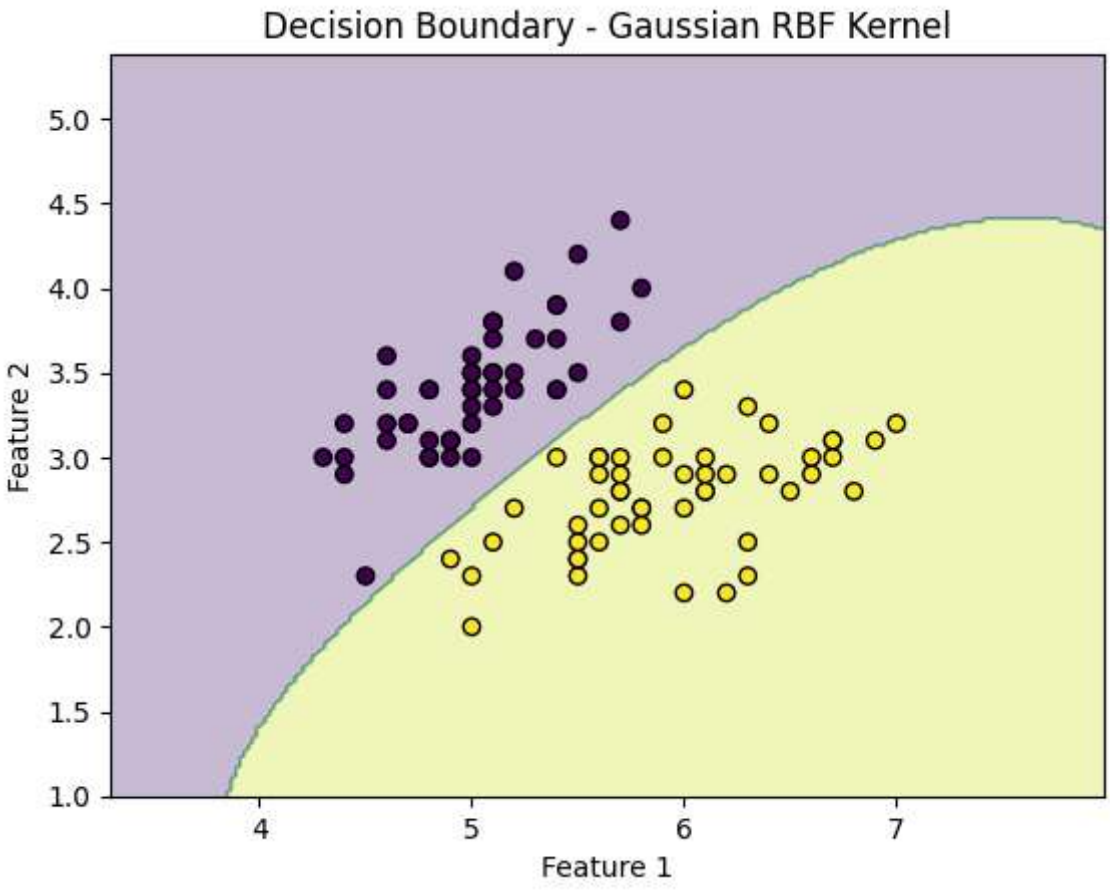
# Decision Boundary
x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
y_min, y_max = X[:,1].min()-1, X[:,1].max()+1

xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                     np.arange(y_min, y_max, 0.02))

Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(X[:,0], X[:,1], c=y, edgecolors="k")
plt.title("Decision Boundary - Gaussian RBF Kernel")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

```
Gaussian RBF Kernel
Accuracy: 1.0
Loss: 0.04872744282047032
```


Decision Boundary - Gaussian RBF Kernel
Feature 2
Feature 1
5.0
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
4
5
6
7

In [ ]:

In [1]:
```
'''
7. Write a ML code for iris datasets classifications using sklearn MLP library
and use the following information to create and train the MLP Classifier.
7a.
*Two hidden layers: the first with 10 neurons, the second with 5 neurons.
 *Number of iteration set to 10, (max_iter=10), random_state=42
* Use 'relu' (Rectified Linear Unit) , it is also the default activation function
for hidden layers.
* use 'adam', and this is also the default solver (optimizer) and is robust for
large datasets.

7b.
*Two hidden layers: the first with 10 neurons, the second with 10 neurons.
 *Number of iteration set to 100, (max_iter=100), random_state=42
* Use 'relu' (Rectified Linear Unit) , it is also the default activation function
for hidden layers.
* use 'adam', and this is also the default solver (optimizer) and is robust for
large datasets.



7c.
*Two hidden layers: the first with 20 neurons, the second with 10 neurons.
 *Number of iteration set to 300, (max_iter=300), random_state=42
* Use 'relu' (Rectified Linear Unit) , it is also the default activation function
for hidden layers.
* use 'adam', and this is also the default solver (optimizer) and is robust for
large datasets.

7d.
*Two hidden layers: the first with 30 neurons, the second with 20 neurons.
 *Number of iteration set to 500, (max_iter=500), random_state=42
* Use 'relu' (Rectified Linear Unit) , it is also the default activation function
for hidden layers.
* use 'adam', and this is also the default solver (optimizer) and is robust for
large datasets.

After implementation of  the given programme, write in your notebook, in our own
sentences, the interpretation of results and explain to TA/JTS  along with
showing programme, and kindly make sure to mark for evaluation of the same.


Note: This time iris data set consists of 3 classes to show the concept of MLP
for multi class classification.
But degree of linearity is more (in future you can experiment with any non-linear
datasets and analyse MLP for classification of non-linear datasets).
'''
```

Out[1]:
```
"\n7. Write a ML code for iris datasets classifications using sklearn MLP library and use the following information to create and train the MLP Classifier.\n7a. \n* Two hidden layers: the first with 10 neurons, the second with 5 neurons.\n *Number of iteration set to 10, (max_iter=10), random_state=42\n* Use 'relu' (Rectified Linear Unit) , it is also the default activation function for hidden layers.\n* use 'adam', and this is also the default solver (optimizer) and is robust for large datasets.\n\n7b. \n*Two hidden layers: the first with 10 neurons, the second with 10 neurons.\n *Number of iteration set to 100, (max_iter=100), random_state=42\n* Use 'relu' (Rectified Linear Unit) , it is also the default activation function for hidden layers.\n* use 'adam', and this is also the default solver (optimizer) and is robust for large datasets.\n\n\n7c. \n*Two hidden layers: the first with 20 neurons, the second with 10 neurons.\n *Number of iteration set to 300, (max_iter=300), random_state=42\n* Use 'relu' (Rectified Linear Unit) , it is also the default activation function for hidden layers.\n* use 'adam', and this is also the default solver (optimizer) and is robust for large datasets.\n\n7d. \n*Two hidden layers: the first with 30 neurons, the second with 20 neurons.\n *Number of iteration set to 500, (max_iter=500), random_state=42\n* Use 'relu' (Rectified Linear Unit) , it is also the default activation function for hidden layers.\n* use 'adam', and this is also the default solver (optimizer) and is robust for large datasets.\n\nAfter implementation of  the given programme, write in your notebook, in our own sentences, the interpretation of results and explain to TA/JTS  along with showing programme, and kindly make sure to mark for evaluation of the same.\n\n\nNote: This time iris data set consists of 3 classes to show the concept of MLP for multi class classification.\nBut degree of linearity is more (in future you can experiment with any non-linear datasets and analyse MLP for classification of non-linear datasets).\n"
```

In [2]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report
```

In [3]:
```
# load dataseyt
iris = load_iris()
X = iris.data
y = iris.target
```

In [4]:
```
# split the dataset into training and testing sets (80`% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
    )
```

In [5]:
```
# function to train and evaluate MLPClassifier with given parameters
def train_and_evaluate(hidden_layer_sizes, max_iter):

    model = MLPClassifier(
        hidden_layer_sizes=hidden_layer_sizes,
        max_iter=max_iter,
        random_state=42,
        activation='relu',
        solver='adam'
    )
```

```
    # Train
    model.fit(X_train, y_train)

    # Predict
    y_pred = model.predict(X_test)

    # Evaluate
    accuracy = accuracy_score(y_test, y_pred)
    report = classification_report(y_test, y_pred)

    return accuracy, report   # ← Must be inside the function
```

In [6]:

```
# Printing results for all configurations

hidden_layer = (30, 20)
max_iter = 100
accuracy, report = train_and_evaluate(hidden_layer, max_iter)

print("Hidden Layers:", hidden_layer)
print("Max Iterations:", max_iter)
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
print("-" * 50)
```

```
Hidden Layers: (30, 20)
Max Iterations: 100
Accuracy: 0.9666666666666667
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      0.89      0.94         9
           2       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30

--------------------------------------------------
```

```
c:\Users\HP\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\neural
_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Ma
ximum iterations (100) reached and the optimization hasn't converged yet.
  warnings.warn(
```

In [ ]:

In [1]:
```
'''8. For the given datasets, use sklearn GDA classification library and find
accuracy for LDA and QDA.
Also plot the class distribution graph to show whether the classes of datasets
follow Gaussian distribution or not.'''
```

Out[1]:
```
'8. For the given datasets, use sklearn GDA classification library and find accura
cy for LDA and QDA.\nAlso plot the class distribution graph to show whether the cl
asses of datasets follow Gaussian distribution or not.'
```

In [2]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
QuadraticDiscriminantAnalysis
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv("iris_dataset_GDA - iris_dataset_GDA.csv")     # Save the given
data as iris_dataset_GDA - iris_dataset_GDA.csv

# Features and Target
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -------- LDA --------
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)
lda_pred = lda.predict(X_test)

# -------- QDA --------
qda = QuadraticDiscriminantAnalysis()
qda.fit(X_train, y_train)
qda_pred = qda.predict(X_test)

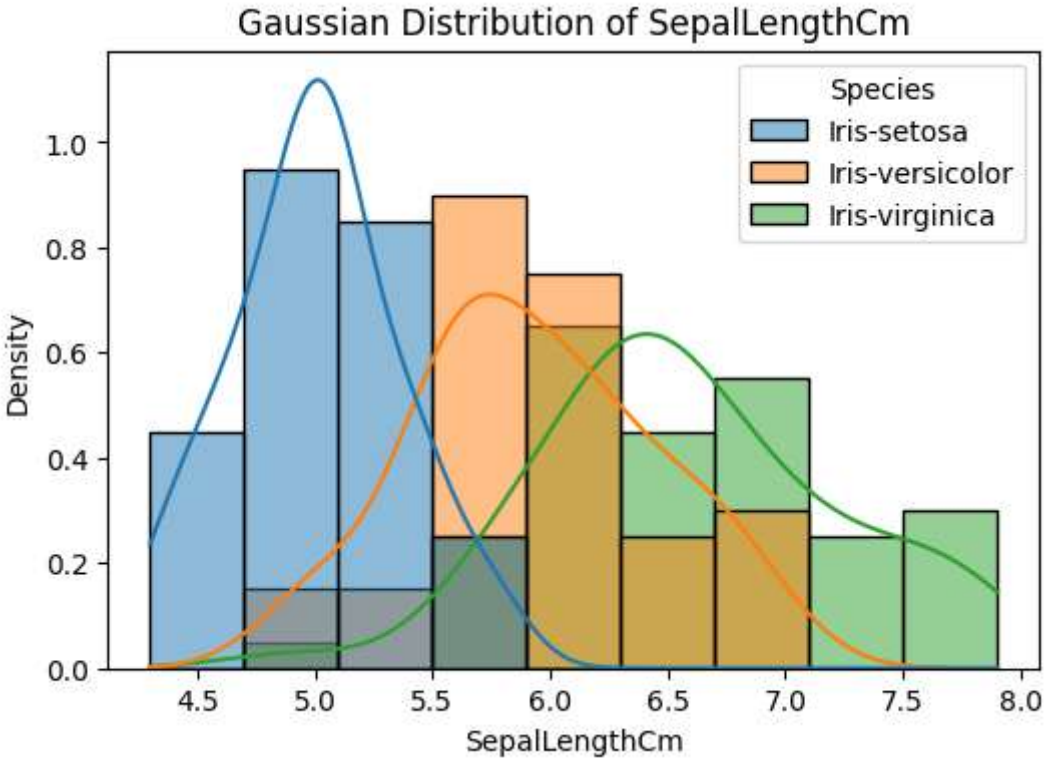
# Accuracy
print("LDA Accuracy :", accuracy_score(y_test, lda_pred))
print("QDA Accuracy :", accuracy_score(y_test, qda_pred))

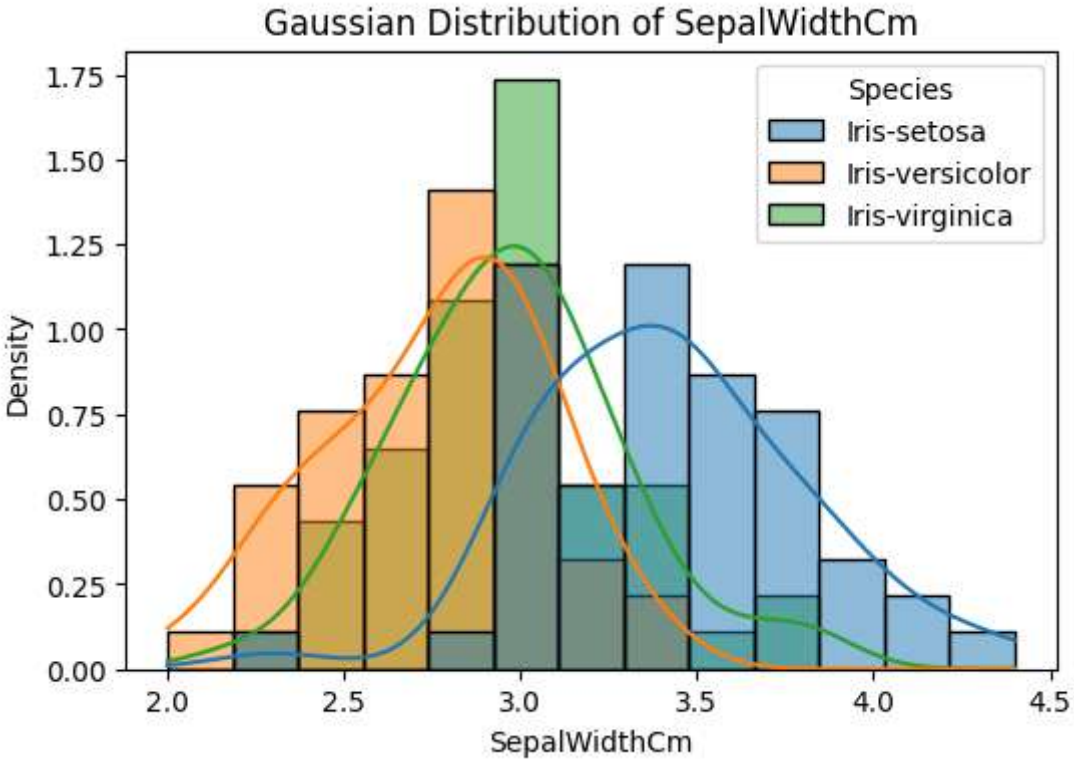
# -------- Gaussian Distribution Plots --------
features = X.columns

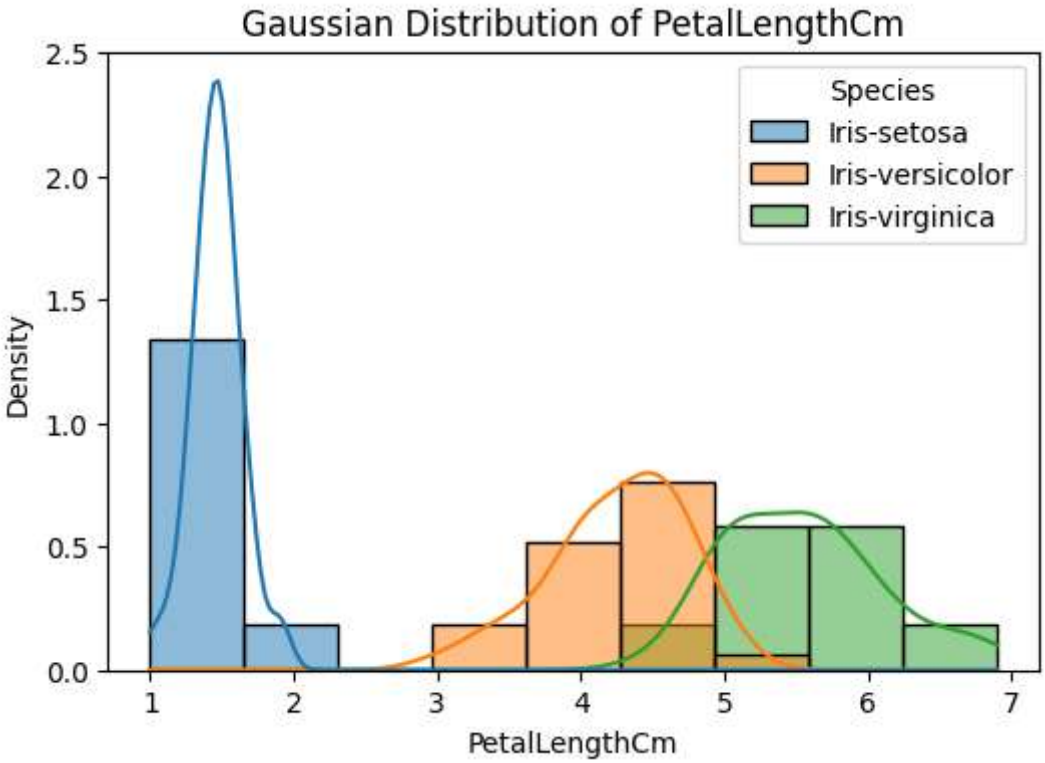
for feature in features:
    plt.figure(figsize=(6,4))
    sns.histplot(
        data=df,
        x=feature,
        hue="Species",
```

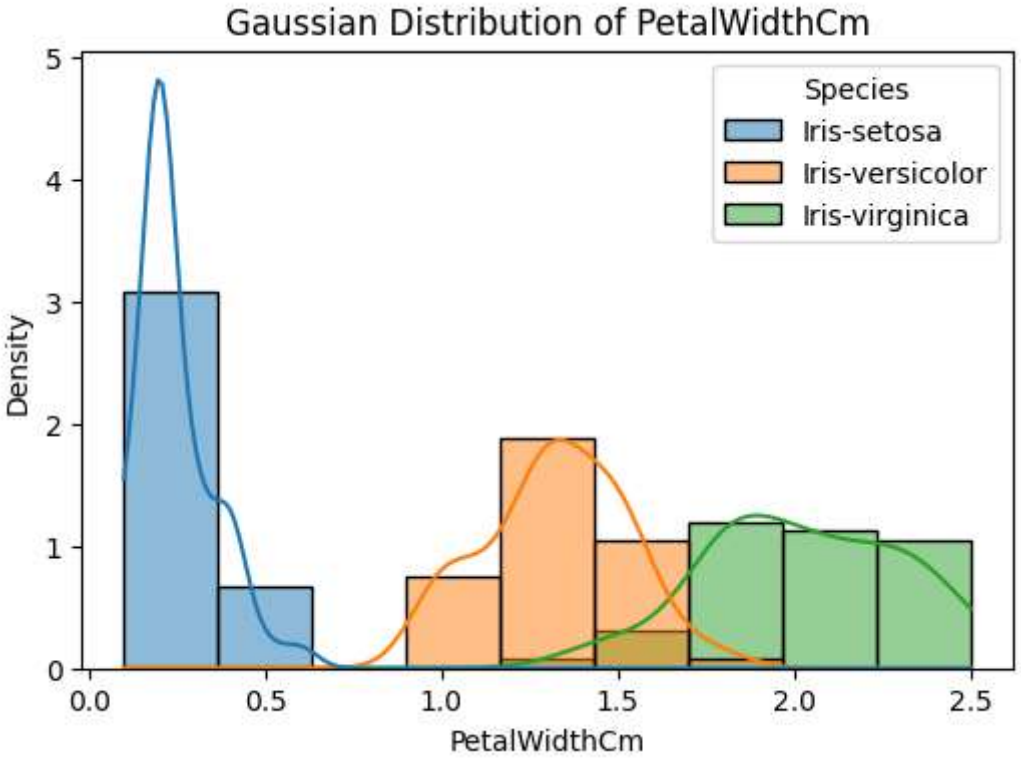
```
        kde=True,
        stat="density",
        common_norm=False
    )
    plt.title(f"Gaussian Distribution of {feature}")
    plt.show()
```

```
LDA Accuracy : 1.0
QDA Accuracy : 0.9666666666666667
```






Gaussian Distribution of PetalLengthCm
Species
Iris-setosa
Iris-versicolor
Iris-virginica
Density
PetalLengthCm
0.0
0.5
1.0
1.5
2.0
2.5
1
2
3
4
5
6
7


Gaussian Distribution of PetalWidthCm
Species
Iris-setosa
Iris-versicolor
Iris-virginica
Density
PetalWidthCm
0
1
2
3
4
5
0.0
0.5
1.0
1.5
2.0
2.5

In [ ]:

In [3]:
```
'''9. Write a ML code for iris classifications using GaussianNB (since datasets
is real numbers (continuous data) and display training and testing accuracy of
your model and classification Report results.

Note: Continuous (real numbers)
Count data (e.g., word frequencies in text)
Binary features (0/1)'''
```

Out[3]:
```
'9. Write a ML code for iris classifications using GaussianNB (since datasets is r
eal numbers (continuous data) and display training and testing accuracy of your mo
del and classification Report results.\n\nNote: Continuous (real numbers)\nCount d
ata (e.g., word frequencies in text)\nBinary features (0/1)'
```

In [4]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
df = pd.read_csv("iris_dataset_NB.csv")      # Save the given data as Iris.csv

# Features and Target
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Split dataset (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Gaussian Naive Bayes Model
model = GaussianNB()
model.fit(X_train, y_train)

# Predictions
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

# Accuracy
train_acc = accuracy_score(y_train, train_pred)
test_acc = accuracy_score(y_test, test_pred)

print("Training Accuracy :", train_acc)
print("Testing Accuracy  :", test_acc)

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, test_pred))
```

```
Training Accuracy : 0.95
Testing Accuracy  : 1.0

Classification Report:

                precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

      accuracy                           1.00        30
     macro avg       1.00      1.00      1.00        30
  weighted avg       1.00      1.00      1.00        30
```

In [ ]:

In [3]:
```
# Write a ML program (using sklearn library) to apply Random Forest Classifier
for given iris dataset and display accuracy,
# confusion matrix and other classification report parameters.
```

In [7]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Load the Iris dataset
data = pd.read_csv("iris_dataset_RF.csv")

# Separate features and target
X = data.drop("Species", axis=1)
y = data["Species"]

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Create and train Random Forest Classifier
model = RandomForestClassifier(
    n_estimators=9, random_state=42
)
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Display accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred, labels=model.classes_))

# Display classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Display individual metrics
from sklearn.metrics import precision_score, recall_score, f1_score

print("Precision:", precision_score(
    y_test, y_pred, average="weighted"
))
print("Recall:", recall_score(
    y_test, y_pred, average="weighted"
))
```

```
print("F1-Score:", f1_score(
    y_test, y_pred, average="weighted"
))
```

```
Accuracy: 0.9666666666666667

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]

Classification Report:
                precision    recall  f1-score   support

   Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      0.90      0.95        10
 Iris-virginica       0.91      1.00      0.95        10

      accuracy                           0.97        30
     macro avg       0.97      0.97      0.97        30
  weighted avg       0.97      0.97      0.97        30

Precision: 0.9696969696969696
Recall: 0.9666666666666667
F1-Score: 0.9665831244778613
```

In [ ]:

In [1]:
```
# 11. First practice the attached AdaBoost ML program and analyse the code and
learn how it works for considered small dataset.
# Implement a AdaBoost ML programme using sklearn AdaBoostClassifier and
DecisionTreeClassifier for attached iris dataset,
# but you need to follow the same code as given above (with minor changes to
include iris dataset) and print
# 1. Print each week learner estimated weight and respective accuracy.
# 2. print test accuracy (final) and classification report (Precision, recall,
f2-score, support values)
```

In [1]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support
)

# 1. Load the Iris dataset
data = pd.read_csv("iris_dataset_Adaboost.csv")

# 2. Separate input features and target
X = data.drop("Species", axis=1)
y = data["Species"]

# 3. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 4. Create AdaBoost with Decision Tree weak learners
base_model = DecisionTreeClassifier(
    max_depth=1,
    random_state=42
)

model = AdaBoostClassifier(
    estimator=base_model,
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)

# 5. Train the model
model.fit(X_train, y_train)

# 6. Print each weak learner's weight, error rate, and accuracy
print("WEAK LEARNER DETAILS")
print("-" * 65)
print(
```

```
    f"{'Learner':<12}"
    f"{'Weight':<15}"
    f"{'Error Rate':<15}"
    f"{'Test Accuracy'}"
)

# for i, learner in enumerate(model.estimators_):
#     # Convert DataFrame to NumPy array to avoid feature-name warnings
#     learner_pred = learner.predict(X_test.to_numpy())

#     weight = model.estimator_weights_[i]
#     error = model.estimator_errors_[i]
#     accuracy = accuracy_score(y_test, learner_pred)

#     print(
#         f"{i + 1:<12}"
#         f"{weight:<15.4f}"
#         f"{error:<15.4f}"
#         f"{accuracy:.4f}"
#     )

# 7. Final AdaBoost prediction
y_pred = model.predict(X_test)

print("\nFINAL ADABOOST RESULTS")
print("-" * 40)
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}")

# 8. Classification report with F2-score
precision, recall, f2, support = precision_recall_fscore_support(
    y_test,
    y_pred,
    labels=model.classes_,
    beta=2,
    zero_division=0
)

report = pd.DataFrame({
    "Precision": precision,
    "Recall": recall,
    "F2-Score": f2,
    "Support": support
}, index=model.classes_)

print("\nCLASSIFICATION REPORT")
print(report.round(4))

# 9. Macro and weighted averages
for avg in ["macro", "weighted"]:
    p, r, f, _ = precision_recall_fscore_support(
        y_test,
        y_pred,
        average=avg,
        beta=2,
        zero_division=0
    )
```

```
    print(f"\n{avg.capitalize()} Average:")
    print(f"Precision: {p:.4f}")
    print(f"Recall:    {r:.4f}")
    print(f"F2-Score:  {f:.4f}")
```

```
WEAK LEARNER DETAILS
--------------------------------------------------------------
Learner     Weight         Error Rate     Test Accuracy

FINAL ADABOOST RESULTS
--------------------------------------
Test Accuracy: 0.9333

CLASSIFICATION REPORT
                 Precision  Recall  F2-Score  Support
Iris-setosa            1.0     1.0       1.0       10
Iris-versicolor        0.9     0.9       0.9       10
Iris-virginica         0.9     0.9       0.9       10

Macro Average:
Precision: 0.9333
Recall:    0.9333
F2-Score:  0.9333

Weighted Average:
Precision: 0.9333
Recall:    0.9333
F2-Score:  0.9333
```

In [ ]:

In [1]:
```
import numpy as np
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score
```

In [2]:
```
# -----------------------------------
# 1. Create Simple Dataset
# -----------------------------------
X = np.array([[1], [2], [3], [4], [5], [6]])
y = np.array([1, -1, 1, -1, 1, -1])   # alternating pattern

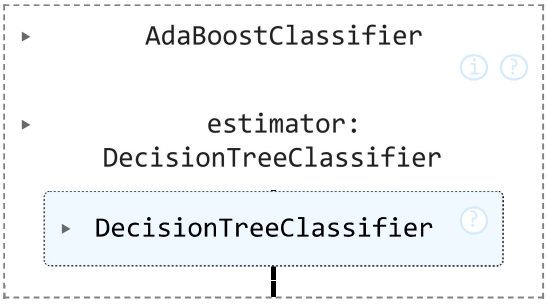
print("Dataset:")
for xi, yi in zip(X.flatten(), y):
    print(f"X={xi}, y={yi}")
```

```
Dataset:
X=1, y=1
X=2, y=-1
X=3, y=1
X=4, y=-1
X=5, y=1
X=6, y=-1
```

In [3]:
```
# -----------------------------------
# 2. Create AdaBoost Model
# -----------------------------------
model = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=3,
    learning_rate=1.0
)
```

In [4]:
```
# -----------------------------------
# 3. Train Model
# -----------------------------------
model.fit(X, y)
```

Out[4]:



In [5]:
```
# -----------------------------------
# 4. Show Each Weak Learner
# -----------------------------------
print("\nWeak Learners:")

for i, (tree, weight) in enumerate(zip(model.estimators_,
```

```
                                        model.estimator_weights_)):
    print(f"\nWeak Learner {i+1}")
    print(export_text(tree, feature_names=["Feature"]))
    print("Alpha (Model Weight):", round(weight, 4))
```

```
Weak Learners:

Weak Learner 1
|--- Feature <= 5.50
|   |--- class: 1
|--- Feature >  5.50
|   |--- class: -1

Alpha (Model Weight): 0.6931

Weak Learner 2
|--- Feature <= 1.50
|   |--- class: 1
|--- Feature >  1.50
|   |--- class: -1

Alpha (Model Weight): 1.0986

Weak Learner 3
|--- Feature <= 5.50
|   |--- class: 1
|--- Feature >  5.50
|   |--- class: -1

Alpha (Model Weight): 0.6931
```

In [6]:
```
# -----------------------------------
# 5. Step-wise Predictions
# -----------------------------------
print("\nStep-wise Accuracy:")

for i, y_pred in enumerate(model.staged_predict(X)):
    acc = accuracy_score(y, y_pred)
    print(f"After learner {i+1}: Accuracy = {round(acc, 3)}")
```

```
Step-wise Accuracy:
After learner 1: Accuracy = 0.667
After learner 2: Accuracy = 0.667
After learner 3: Accuracy = 0.667
```

In [7]:
```
# -----------------------------------
# 6. Final Strong Model
# -----------------------------------
final_pred = model.predict(X)
final_acc = accuracy_score(y, final_pred)

print("\nFinal Predictions:", final_pred)
print("Final Accuracy:", final_acc)
```

```
Final Predictions: [ 1  1  1  1  1 -1]
Final Accuracy: 0.6666666666666666
```